The Oracle Forge

Context Engineering & Evaluation Science for Production Data Agents

Weeks 8 – 9 · Team Challenge

April 2026

The Mission

In March 2026, the source code of Claude Code — Anthropic's autonomous coding agent — was accidentally published to npm. Within hours, 512,000 lines of TypeScript were mirrored, forked, and studied by engineers worldwide. It was the first time the internal architecture of a frontier AI agent harness was available for systematic study.

You are going to study it, learn from it, and build something with those lessons.

Your team's mission across Weeks 8 and 9 is to design and deliver a production-grade data analytics agent — one that applies engineering principles from the Claude Code architecture, context-layering patterns from OpenAI's internal data agent, and evaluation methodology from the UC Berkeley DataAgentBench (DAB) research community. The agent must run against real multi-database workloads, produce verifiable outputs, and compete on a public benchmark leaderboard.

The measure of success is whether the thing you built would have been inconceivable to build a year ago — not because the data is hard, but because the engineering discipline required to make it reliable did not yet exist as a teachable practice.

What You Have Already Built

This challenge does not start from zero. By Week 8, your cohort has built six interconnected systems across the previous arcs. The Oracle Forge is the integration test — the moment where those systems are called upon to work together as components of something larger.

Previous project	What it produced	How it connects here
Week 2 — Automaton Auditor	A hierarchical multi-agent code evaluation system with structured scoring	The evaluation harness for your data agent follows the same pattern: structured output, multi-layer assessment, measurable improvement between runs

Week 3 — Document Intelligence Refinery	An agentic pipeline for extracting structured data from unstructured documents	DataAgentBench explicitly requires "unstructured text transformation" as a capability — your Week 3 pipeline is the starting point
Week 4 — Brownfield Cartographer	A multi-agent codebase intelligence system with lineage graphs and Claude Code worktrees	The agent harness architecture here mirrors the Conductor/worktree pattern you built; the tenai-infra system you use for infrastructure is the same one used in Week 4
Week 5-6 — The Ledger: Event Sourcing	An event-sourcing system is designed to build the immutable memory and governance backbone required for multi-agent AI systems to move into production at scale	You are not building the evaluation harness from scratch—you are adapting what you built in Week 5 for a new problem domain. This harness applies the same trace schema, scoring design, and regression detection principles
Week 7 — Data Contract Enforcer	A schema validation and data quality enforcement system with lineage-based attribution	Your agent must enforce data contracts on its own outputs — every result returned to the user is implicitly a contract claim that needs validation before delivery

> **The integration test:** If your previous work was a collection of tools, this challenge is the moment they become a system. Trainees who treat prior weeks as disconnected exercises will struggle here. Trainees who treat them as components of a larger machine will surprise themselves with what is possible in two weeks.

What You Are Building

The agent you build is a natural language data analyst capable of answering complex business questions against heterogeneous databases — the kind that real enterprise environments actually contain, not the clean single-table demos that benchmarks usually hide behind.

A user asks: "Which customer segments had declining repeat purchase rates in Q3, and does that pattern correlate with the support ticket volume in our CRM?" Your agent must navigate two databases (a transaction DB and a CRM DB), resolve inconsistently formatted customer IDs across them, extract structured data from unstructured support notes, and produce a verifiable answer with an auditable query trace.

That is not a hard question for a skilled analyst. It is an extremely hard question for an AI agent to answer reliably, transparently, and in a way that can be measured and improved. The engineering that closes this gap is what this challenge is about.

The Three Engineering Challenges

Challenge 1 — Multi-layer context architecture

The Claude Code leak and the OpenAI data agent writeup converge on the same insight: the bottleneck in production data agents is not query generation — it is context. An agent that cannot find the right table across multiple databases, understand what a business term means in this organisation's data, or remember what the user corrected it on in the last session, will fail on questions trivially easy for a human analyst.

Your agent must implement at minimum three context layers: (a) schema and metadata knowledge across all connected databases, populated before the agent answers its first question; (b) institutional and domain knowledge — what "revenue" means in this dataset, which tables are authoritative versus deprecated, how customer IDs are formatted differently across systems; and (c) interaction memory — corrections received, successful query patterns, user preferences across sessions. The OpenAI data agent uses six layers. You need at minimum three that demonstrably work.

Challenge 2 — Self-correcting execution across heterogeneous databases

DataAgentBench's defining property is that it requires agents to work across multiple database systems — PostgreSQL, MongoDB, SQLite, DuckDB — in the same query session. This is the norm in enterprise environments and the reason why most data agents that work in demos fail in production. Your agent must detect execution failures, diagnose whether the failure is in the query, the join key format, the database type, or the data quality, and recover without surfacing the error to the user.

Challenge 3 — Evaluation harness with measurable improvement

You cannot compete on a benchmark if you cannot tell whether your last change made the agent better or worse. Your evaluation harness must trace every tool call, record query outcomes against expected results, and produce a score that improves measurably between Week 8 and Week 9. This harness is The Sentinel applied to data agents. The trace schema is the same. The scoring design follows the same principles. The regression detection is the same. You are not building it from scratch — you are adapting what you built in Week 5 for a new problem domain.

The Benchmark — DataAgentBench

DataAgentBench (DAB) is the first benchmark that evaluates AI data agents on realistic enterprise workloads. It was produced by the UC Berkeley EPIC Data Lab in collaboration with PromptQL (Hasura) and published in 2026. It is the right benchmark for this challenge because it tests exactly what real enterprise data agents face — not clean single-table text-to-SQL, but the messy, multi-database, semantically ambiguous queries that make production data agents hard.

Property	DAB specification
Total queries	54 queries across 12 datasets
Domains covered	9 domains including retail, telecom, healthcare, finance, anti-money laundering

Database systems	4: PostgreSQL, MongoDB, SQLite, DuckDB — often multiple in the same query
Key challenges tested	Multi-database integration, ill-formatted join keys, unstructured text transformation, domain knowledge requirements
Best current score	PromptQL + Gemini 3.1 Pro: 54.3% pass@1 — substantial room to compete
Evaluation method	Run agent on all 54 queries, n ≥ 50 trials per query, submit results JSON via GitHub PR
Contribution opportunity	Teams can submit new queries from their own data work to extend the benchmark

The 38% ceiling for the best frontier model is not a flaw in the benchmark — it is a signal about the gap between raw LLM capability and the engineering required to make a reliable data agent. A well-engineered system with proper context layering, multi-pass retrieval, and self-correcting execution can close that gap significantly. That is your target.

DAB's four hard requirements

- **Most DAB queries require joining or reconciling data across two or more database systems. Your agent must route queries to the correct database, handle different query dialects (SQL vs. MongoDB aggregation pipelines vs. DuckDB analytical SQL), and merge results correctly.**Multi-database integration:
- **Customer IDs, product codes, and entity references are inconsistently formatted across databases — the norm in enterprise data, almost never present in academic benchmarks. Your agent must resolve these without assuming a clean schema.**Ill-formatted join keys:
- **Some queries require extracting structured facts from unstructured text fields — support notes, product descriptions, free-text comments. This is where Week 3's Document Intelligence work directly applies.**Unstructured text transformation:
- **Answering correctly requires knowing things that are not in the schema — what "churn" means in this industry, what the fiscal year boundaries are, which status codes indicate active versus inactive accounts. This is what the Knowledge Base built by Intelligence Officers is for.** Domain knowledge:

> **Contribution track:** Teams that build an agent capable of answering queries not currently in DAB can submit new query-answer pairs to the benchmark repository. This is a genuine research contribution — the first time a cohort from this program appears in a UC Berkeley benchmark dataset.

Team Structure and Roles

Your cohort is organised into seven teams of six. Each team operates with three roles. The roles are not silos — understanding across roles is expected, and knowledge compounds when members teach each other. Primary accountability for each deliverable belongs to one role, but the work is done together.

Missing any one role produces a predictable failure mode: Drivers without Intelligence Officers build in the dark. Intelligence Officers without Drivers produce knowledge that never ships. Both without Signal Corps produce work the world never sees.

Role 1 — Drivers (2 members)

Drivers hold the primary accountable role for the running codebase. They are not the only ones who interact with the AI — but they hold the driving seat during mob sessions, meaning they control the keyboard and AI tool interface while the rest of the team actively co-pilots.

Before each sprint, Drivers produce an Inception document using the AWS AI-DLC framework. AI-DLC organises development across three phases: Inception (define what will be built and why), Construction (build it under AI assistance with mob discipline), and Operations (verify it works and document what happened). A human approval gate separates each phase — the team must explicitly agree that Inception is complete before Construction begins, and that Construction is complete before Operations begins. This is not a process for its own sake. It is the mechanism that prevents AI tools from accelerating a team past the point where anyone actually understands what has been built.

The framework is documentation-first: if the entire codebase were deleted, the Inception documents, construction logs, and operations records would survive intact. The complete record of what was built and why remains legible without a running system.

Driver deliverable	Description
AI-DLC Inception document (one per sprint)	Three sections: what is being built (press-release style, one paragraph), what could go wrong (honest FAQ), what done looks like (numbered, verifiable checklist). Approved by the full team before Construction begins.
Running agent on shared server	Deployed on the tenai-infra system, accessible to all team members from any device. Functional end of Week 8.
Evaluation harness with documented test results	Sentinel-pattern trace log, quality scores against held-out test set, regression suite. Score log showing improvement from first run to benchmark submission.
Benchmark submission	Agent run against all 54 DAB queries, n ≥ 50 trials each, results JSON submitted via GitHub PR to the DAB repository.
Codebase in team repository	Documented, reproducible from a clean clone. Any team member can re-deploy from the README.

Role 2 — Intelligence Officers (2 members)

Intelligence Officers are the team's knowledge function. They navigate the global research and engineering ecosystem — benchmark papers, the Claude Code architecture analyses, community

discussions, open-source implementations — and convert what they find into structured, injectable knowledge that makes the entire team more effective.

The primary output of an Intelligence Officer is the team's LLM Knowledge Base, built using the Karpathy method: a minimal set of precisely structured documents that can be loaded directly into an LLM context window to give it working knowledge of a specific domain. "Minimal and precise" is the discipline — documents that grow without being verified become noise. The Intelligence Officer tests every KB document by injecting it into a fresh context and confirming it produces correct outputs.

Intelligence Officers also build shared utilities — reusable modules that any team member can use. The adversarial probe library is their other core deliverable: a structured set of queries designed to expose specific failure modes in the agent, covering each of DAB's four hard requirement categories.

Intelligence Officer deliverable	Description
LLM Knowledge Base v1 — architecture	Markdown documents covering: Claude Code three-layer memory system, autoDream consolidation, tool scoping; OpenAI data agent six-layer context design. Each document is verified by an injection test. Ready before Drivers write their first agent code.
LLM Knowledge Base v2 — domain	Schema descriptions for DAB's databases, known query patterns that work, ill-formatted join key glossary, unstructured field inventory, domain term definitions. Updated as Drivers discover new patterns.
LLM Knowledge Base v3 — corrections log	Running structured log of agent failures: [query that failed] → [what was wrong] → [correct approach]. Written by Drivers after every observed failure, maintained by Intelligence Officers.
Shared utility library	At least three reusable, documented modules: examples include multi-pass retrieval helper, schema introspection tool, benchmark harness wrapper, join key resolver. Committed with tests.
Adversarial probe library	At least 15 documented probes across at least 3 of DAB's four failure categories. Each probe documents the query, expected failure mode, observed agent response, and fix that worked.
Weekly global ecosystem report	One page: what the research and engineering community is doing on data agents that the team should know. Presented at the Monday mob session.

Role 3 — Signal Corps (2 members)

Signal Corps carries the team's work into the world and brings the world's attention back. They are not producing PR for a finished product — they are documenting a live engineering process in real time and participating in the global community that is working on the same problems your team is working on.

The work of Signal Corps requires technical depth. You cannot write a credible thread about multi-database context engineering if you do not understand what it means. You cannot participate in

the DAB benchmark community if you have not read the paper. Signal Corps members are expected to attend mob sessions, understand the technical progress, and represent it accurately externally.

Signal Corps deliverable	Description
Internal Slack daily post	One post per day: what shipped, what is stuck, what is next. Four bullet points maximum. The facilitator's primary visibility into team progress.
X threads (minimum 2 per week)	Technical threads engaging with notable posts about data agents, Claude Code architecture, or DAB benchmark work. Substantive: add something, ask something, answer something. Not announcements.
LinkedIn article or Medium post	One substantive technical post per Signal Corps member across the two weeks. Minimum 600 words. Topic: one specific thing learned, one failure understood, one engineering decision made. Not a project summary.
Community participation log	Links to Reddit (r/MachineLearning, r/LocalLLaMA), Discord (Hugging Face, DAB community), X threads where the team engaged with a substantive comment. Updated daily.
Resource acquisition report	Applications made for free-tier resources (Cloudflare Workers, API credits, compute access), outcomes, and instructions for the team to use what was obtained.
External engagement summary	Compiled at end of Week 9: all post links, reach metrics where available, notable responses received, any community intelligence that changed the team's technical approach.

The Mob Session — How the Team Works

The primary working unit of this challenge is the daily mob session. Every day, the full team works together for at least one hour using mob programming discipline. One Driver holds the keyboard and AI tool interface. Everyone else is an active co-pilot.

In mob programming, the driver executes — but only what the team decides. Questions are asked aloud. AI-generated questionnaires are answered by the group. Approval gates between AI-DLC phases happen at mob sessions: the team reads the Inception document together, asks the hardest questions they can think of, and gives or withholds approval together. No phase transition happens in isolation.

The tenai-infra system is designed for this. With Tailscale mesh networking, all team members can connect from any device — laptop, mobile, tablet — to shared tmux sessions on the team server. The Gemini CLI conductor manages parallel agent sessions that any team member can observe and direct. Git worktrees allow multiple experiments to run simultaneously without interfering with each other. A mob session running on tenai-infra is not a video call with shared screen — it is a live multi-device engineering environment where all participants can see and act on the same state.

Daily mob ritual: Minimum one hour, every working day. Structure: (10 min) Intelligence Officers present what is new in the Knowledge Base and global ecosystem. (10 min) Signal Corps report what went out and what response it got. (40 min) Drivers lead construction or operations, full team co-piloting. All approval gates for AI-DLC phase transitions happen here, not in Slack.

Compound Engineering Across Two Weeks

Compound engineering is the principle that the outputs of earlier work become inputs to later work, creating value that multiplies rather than accumulates. Six people working in parallel for two weeks produces six independent outputs. Six people working with compound methodology produce a system where each person's work makes every other person's work more powerful.

The compounding in this challenge runs across three axes:

- **Intelligence Officers' Knowledge Base is injected into the Drivers' agent context. Every agent failure that Drivers log becomes a correction entry that Intelligence Officers add to KB v3. The agent improves not because the model improved but because the context it has access to improved.**Knowledge compounds upward:
- **Utilities built by Intelligence Officers are used by Drivers in the implementation. The evaluation harness built by Drivers is used by Signal Corps to report benchmark progress accurately. Nothing is built twice.**Code compounds outward:
- **Signal Corps's external posts attract community attention — from the DAB benchmark team, from practitioners building similar systems, from other trainees in adjacent programmes. That attention sometimes brings back technical insights that change what the team builds.**Visibility compounds the work:

For the compounding to happen, three things are required that most teams skip: a shared repository with actual documentation (not just code), a daily communication rhythm that keeps all three roles informed of each other's current state, and a weekly synthesis meeting where the Knowledge Base is reviewed, the harness score is updated, and the next sprint is planned together.

What to Study

This challenge is built on four bodies of existing work. Intelligence Officers should have the key findings from all four areas in the Knowledge Base before Drivers write their first line of agent code. Drivers should have read the summaries. Signal Corps should understand enough to represent the ideas accurately externally.

Source	What to extract	Where
Claude Code source leak (March 2026)	Three-layer MEMORY.md architecture (index → topic files → session transcripts), autoDream memory consolidation pattern,	github.com/sanbuphy/claude-code-source-code (docs/en/),

	tool scoping philosophy (40+ tools with tight domain boundaries), fork/worktree sub-agent spawn modes	github.com/chauncygu/collection-claude-code-source-code
OpenAI in-house data agent (Jan 2026)	Six-layer context architecture, Codex-powered table enrichment as the hardest sub-problem ("70,000 tables"), self-learning memory loop, the "closed-loop self-correction" pattern	openai.com/index/inside-our-in-house-data-agent
DataAgentBench paper (2026)	The four hard requirements (multi-DB, ill-formatted keys, unstructured text, domain knowledge), failure mode taxonomy from the five model evaluations, the 38% ceiling and what causes failures	arxiv.org/html/2603.20576, github.com/ucbepic/DataAgentBench
AI Agent Internals Strategy (this package)	Five probe types for exposing how any AI agent handles context, the comparative interrogation method, capability matrix template for documenting findings from probing Claude/ChatGPT/Gemini	AI_Agent_Internals_Strategy.md in this challenge package

Combined Deliverables

All deliverables are submitted as a single team package. There are no individual submissions. Evaluation covers the integrated product, not isolated components. A brilliant evaluation harness attached to a non-running agent is not a passing submission.

Deliverable	Owner	Weight
Running agent on shared server (live demo accessible to facilitator)	Drivers	25%
Benchmark submission — DAB results JSON via GitHub PR with recorded score	Drivers	20%
LLM Knowledge Base v1, v2, v3 with changelogs and injection test evidence	Intelligence Officers	15%
Evaluation harness with score log showing measurable improvement	Drivers + Intelligence Officers	10%
Adversarial probe library — 15+ probes, 3+ failure categories, fix documentation	Intelligence Officers	10%
Signal Corps engagement portfolio — all posts, threads, articles, community log	Signal Corps	10%
AI-DLC Inception documents for all sprints with team approval records	Drivers	5%

Shared utility library — documented, tested, reusable by any team member	Intelligence Officers	5%

Interim Submission: Wednesday, April 15 - 21:00 UTC

Submit: GitHub repo + PDF report (public Google Drive link)

This deadline covers infrastructure, core agent build, knowledge base foundations, and initial evaluation; essentially everything through end of Week 8 and into early Week 9.

GitHub repo must contain:

1. **README.md** at root — team members and roles, architecture diagram (hand-drawn and photographed is fine), setup instructions a facilitator can follow on a fresh machine, link to live agent on shared server.

2. **agent/ directory** — AGENT.md context file, MCP tools.yaml with connections to all four database types (PostgreSQL, SQLite, MongoDB, DuckDB), all agent source files, requirements.txt or equivalent. Agent must be running on the shared server, handling at least two DAB database types, with basic NL-to-query working.

3. **kb/ directory** — Four subdirectories (architecture, domain, evaluation, corrections), each with a CHANGELOG.md and evidence of injection tests (test queries and expected answers used to verify each document). Minimum KB v1 (architecture) and KB v2 (domain) committed.

4. **eval/ directory** — Evaluation harness source, initial score log showing at least a first-run baseline against the held-out set. Harness must produce scores with query trace.

5. **planning/ directory** — AI-DLC Inception document(s) with mob session approval records (date, who approved, hardest question asked and its answer). Inception must be team-approved before any Construction code was written.

6. **utils/ directory** — Shared utility library (minimum 3 reusable modules) with README describing each module, usage example, and test.

7. **signal/ directory** — engagement_log.md with all Week 8 post links and any available metrics, community participation log with substantive comment links. Minimum: first X thread live, daily Slack posts, at least one Reddit/Discord substantive comment.

PDF report must cover:

- Architecture overview and key design decisions from the Inception document
- Infrastructure status: tenai-infra running(optional), Tailscale mesh verified(optional), DAB databases loaded, MCP Toolbox configured
- Knowledge Base status: which documents exist, injection test results
- Evaluation harness baseline score and methodology
- Signal Corps Week 8 engagement summary (posts, threads, community intelligence gathered)
- Mob session log summaries showing AI-DLC gate approvals
- What is working, what is not, what the plan is for the remaining days

Final Submission: Saturday, April 18 - 21:00 UTC

Submit: GitHub repo + PDF report + Demo Video (all public, no login required)

This deadline is the final submission — everything from Wednesday plus adversarial testing, benchmark submission, published articles, and the complete engagement portfolio.

GitHub repo adds to Wednesday's:

8. **probes/ directory** — probes.md with all 15+ adversarial probes in standard format: query, failure category (multi-database routing, ill-formatted key mismatch, unstructured text extraction, domain knowledge gap), expected failure, observed failure, fix applied, post-fix score. Minimum 3 of the 4 failure categories covered.

9. **results/ directory** — DAB results JSON (54 queries, 50 trials each), PR link to the DataAgentBench repository, harness score log showing measurable improvement from first run to final submission, screenshot of leaderboard entry if available.

10. **kb/ directory updated** — KB v3 (corrections log) populated with structured failure entries: [query that failed] → [what was wrong] → [correct approach]. Corrections log must demonstrably change agent behaviour on repeated queries.

11. **eval/ directory updated** — Score log showing progression from first run to final (minimum two data points), regression suite running clean against held-out test set.

12. **signal/ directory updated** — Article text files (or links) for each Signal Corps member's published LinkedIn/Medium post (minimum 600 words each), complete engagement portfolio with all post links, reach metrics, notable responses, and community intelligence that changed the team's technical approach. Week 9 content: benchmark X thread, published articles, final community thread, DAB PR announcement.

13. **Benchmark submission** — GitHub PR opened to ucbepic/DataAgentBench with results JSON and AGENT.md. PR title: "[Team Name] — TRP1 FDE Programme, April 2026". Includes pass@1 score, trial count, and brief architecture description.

PDF report (final version) must cover:

- Everything from Wednesday's report, updated
- Final benchmark score and comparison to baseline (measurable improvement required)
- Adversarial probe library summary: failure categories tested, fixes applied, score impact
- KB v3 corrections log impact: how the self-learning loop improved agent performance
- Context engineering depth: all three context layers documented with evidence they work
- Complete Signal Corps engagement portfolio summary
- AI-DLC Operations document: what was built, what changed from the plan, what the harness score is, what the next sprint's Inception should address
- Honest retrospective: what compounded across roles, what didn't, what would change

Demo Video (max 8 minutes):

- Live demonstration of the agent running on the shared server, answering at least 2 DAB queries that span different database types
- Show the self-correction loop: a query that fails, the agent diagnosing and recovering
- Show context layers in action: the agent using institutional knowledge from the KB to resolve an ambiguous term or ill-formatted join key
- Show the evaluation harness producing a score with query trace
- Brief walkthrough of the adversarial probe library and how a probe led to a fix
- No login required to view — host on YouTube (unlisted is fine) or public Google Drive

Evaluation Rubric

Assessment runs across four dimensions. Solid on all four is a pass. Advanced on at least two is distinguished. A team that submits to the benchmark and scores below the current state of the art has still demonstrated more than a team that did not submit — the harness that produced the score is the real deliverable.

Dimension	Solid	Advanced
Technical depth	Agent runs against DAB databases, handles at least two database types, self-corrects on execution failure, produces verifiable output with query trace	Agent handles all four DAB database types; score improves measurably between first run and benchmark submission as evidenced by harness log
Context engineering quality	Three context layers implemented and populated; Knowledge Base injected into agent context; correction log read and written	Agent demonstrably uses institutional knowledge to resolve cross-database entity mismatches; KB changelog shows Synthesizer-Driver feedback producing score improvement
Evaluation science	Harness produces scores, traces tool calls, runs regression suite against held-out queries	DAB benchmark submission with score recorded; adversarial probe library covers multi-DB, unstructured text, and domain knowledge failure categories
Team compounding evidence	All three roles active with deliverables; shared repo has commits from all members; Slack log shows cross-role communication	Signal Corps output directly references team's technical findings with accuracy; KB v3 corrections log shows observable agent improvement; mob session log documents AI-DLC gate approvals